Multiway Cut and k-Cut

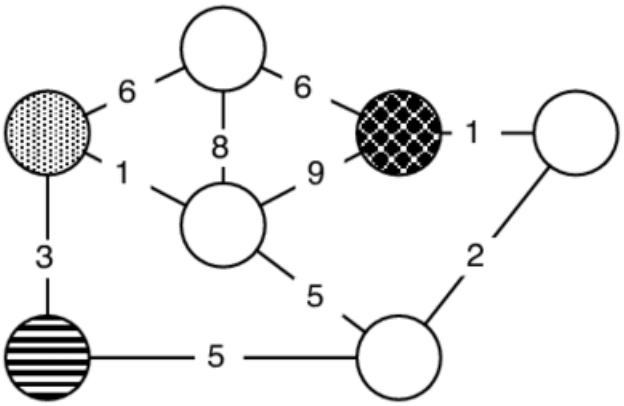
December 7, 2025

Contents

1 Multiwaycut

1.1 Definition(Multiwaycut)

Given a set of terminals $S = s1, s2, ..., sk \subseteq V$, a multiway cut is a set of edges whose removal disconnects the terminals from each other. The multiway cut problem asks for the minimum weight suchset.



1.2 Definition(Max-flow problem)

Maximum flow problems involve finding a feasible flow through a flow network that obtains the maximum possible flow rate:

First we establish some notation:

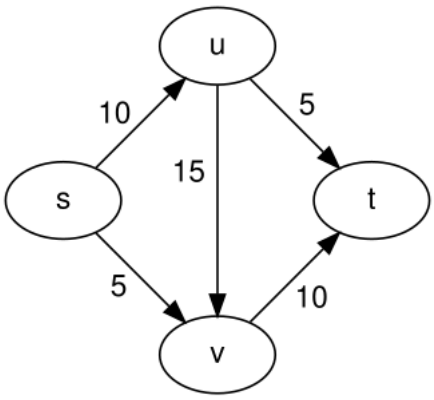
- Let $N = (V, E)$ be a flow network with $s, t \in V$ being the source and the sink of N
- If g is a function on the edges of N then its value on $(u, v) \in E$ is denoted by g_{uv}or $g(u, v)$.

Definition: The capacity of an edge is the maximum amount of flow that can pass through an edge. Formally it is a map $c : E \to \mathbb{R}^+$.

Definition. A flow is a map $f : E \to R$ that satisfies the following:

- Capacity constraint. The flow of an edge cannot exceed its capacity, in other words: $f_{uv} \leq c_{uv}$for all $(u, v) \in E$.
- Conservation of flows. The sum of the flows entering a node must equal the sum of the flows exiting that node, except for the source and the sink. Or: $\forall v \in V \setminus \{s, t\} : \sum_{u:(u,v)\in E, f_{uv}>0} f_{uv} = \sum_{u:(v,u)\in E, f_{vu}>0} f_{vu}$.

Definition. The value of flow is the amount of flow passing from the source to the sink. Formally for a flow $f : E \to \mathbb{R}^+$given by: $|f| = \sum_{v:\ (s,v)\in E} f_{sv} = \sum_{u:\ (u,t)\in E} f_{ut}$.



1.3 Definition(isolating cut)

Define an isolating cut for s_i to be a set of edges whose removal disconnectss_i from the rest of the terminals.

1.4 Observation

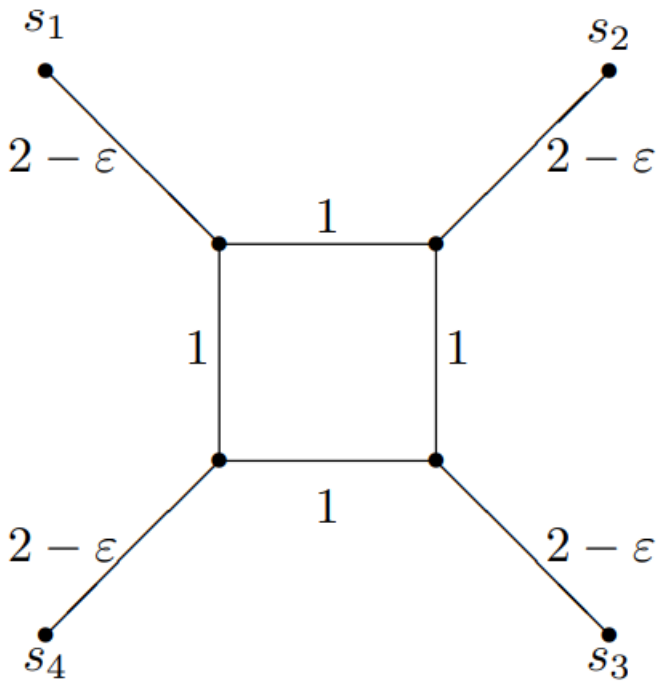
If you set up a flow network with one terminal $s_i$as source, and a sink that (directly or via some gadget) represents "all other terminals or rest of graph," then computing max-flow/min-cut yields a minimum s-i-vs-rest cut. That is precisely an isolating cut for s_i.

1.5 Algorithm(max-flow/s-t min cut)

Cuius rei algorithmum mirabilem sane detexi. Hanc marginis exiguitas non caperet.

1.6 Algorithm(Multiway cut)

1. For each $i = 1, ..., k$, compute a minimum weight isolating cut for s_i, say C_i.
2. Discard the heaviest of these cuts, and output the union of the rest, say C.



1.7 Theorem

Algorithm 1.6 achieves an approximation guarantee of $2 - \frac{2}{k}$.

Demostration: Let A be an optimal multiway cut in G. We can view A as the union of k cuts as follows: The removal of A from G will create k connected components, each having one terminal (since A is a minimum weight multiway cut, no more than k components will be created). Let A_i be the cut separating the component containing si from the rest of the graph. Then $A = \cup_{i=1}^{k} A_i$.

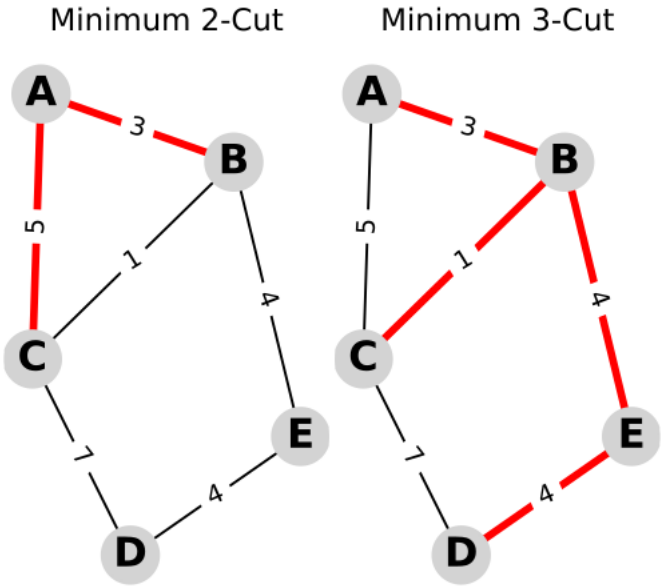
Since each edge of A is incident at two of these components, each edge will be in two of the cuts A_i. Hence, $\sum_{i=1}^{k} w(A_i) = 2w(A)$, where $w(e)$ stands for the weight for edge e.

Clearly, A_i is an isolating cut for s_i. Since C_i is a minimum weight isolating cut for s_i, $w(C_i) \leq w(A_i)$. Notice that this already gives a factor 2 algorithm, by taking the union of all k cuts C_i. Finally, since C is obtained by discarding the heaviest of the cuts C_i, $w(C) \leq (1 - \frac{1}{k}) \sum_{i=1}^{k} w(C_i) \leq (1 - \frac{1}{k}) \sum_{i=1}^{k} w(A_i) = 2(1 - \frac{1}{k})w(A)$

2 Minimum k-cut

2.1 Definition(Minimum k-cut)

A set of edges whose removal leaves k connected components is called a k-cut. The k-cut problem asks for a minimum weight k-cut.



2.2 Observation

A natural algorithm for finding a k-cut is as follows. Starting with G, compute a minimum cut in each connected component and remove the lightest one; repeat until there are k connected components.

2.3 Definition(Gomory–Hu trees)

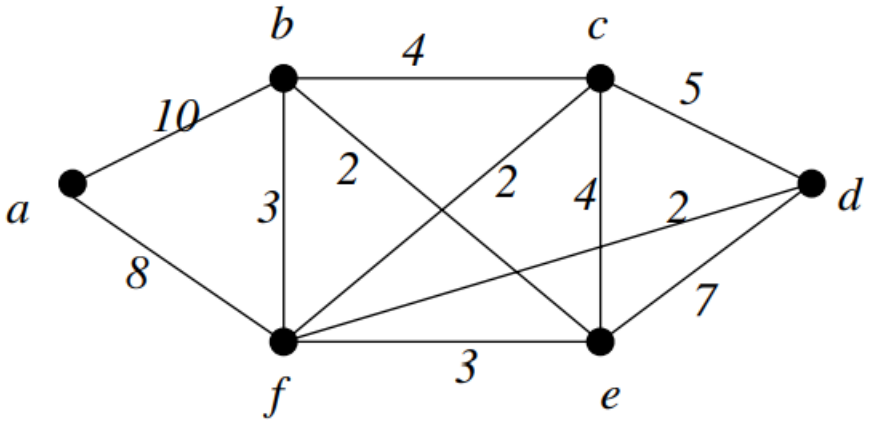
Let $G = (V_G, E_G, c)$be an undirected graph with $c(u, v)$ being the capacity of the edges (u, v) respectively.

Denote the minimum capacity of an s-t cut by $\lambda_{st}$for each $s, t \in V_G$.

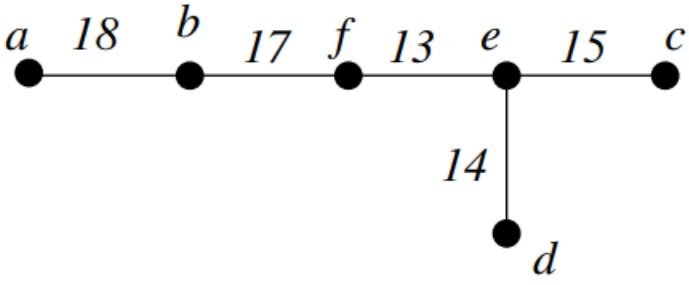
Let $T = (V_G, E_T)$be a tree, and denote the set of edges in an s-t path by $P_{st}$for each $s, t \in V_G$.

Then T is said to be a Gomory–Hu tree of G, if for each $s, t \in V_G$, has$\lambda_{st} = \min_{e \in P_{st}} c(S_e, T_e)$, where

1. $S_e, T_e \subseteq V_G$are the two connected components of $T \setminus \{e\}$, and thus (S_e, T_e) forms an s-t cut in G.
2. $c(S_e, T_e)$ is the capacity of the (S_e, T_e) cut in G.



The graph G



The graph T

2.4 Lemma

Let S be the union of cuts in graph G associated with l edges from graph T. Then, the removal of S from G leaves a graph with at least $l+1$ components.

Demostration: Trivial

2.5 Algorithm(Minimum k-cut)

1. Compute a Gomory–Hu tree T for G.
2. Output the union of the lightest $k-1$ cuts of the $n-1$ cuts associated with edges of T in G; let C be this union.

2.6 Theorem

Algorithm 2.5 achieves an approximation factor of $2 - \frac{2}{k}$.

Demostration: Let A be an optimal k-cut in G. As in Theorem 1.7, we can view A as the union of k cuts: Let $V_1, V_2, ..., V_k$ be the k components formed by removing A from G, and let A_i denote the cut separating V_i from the rest of the graph. Then $A = A_1 \cup \cdots \cup A_k$, and, since each edge of A lies in two of these cuts, $\sum_{i=1}^{k} w(A_i) = 2w(A)$

Without loss of generality assume that A_k is the heaviest of these cuts. The idea behind the rest of the proof is to show that there are $k-1$ cuts defined by the edges of T whose weights are dominated by the weight of the cuts $A_1, A_2, ..., A_{k-1}$. Since the algorithm picks the lightest $k-1$ cuts defined by T, the theorem follows. Thus we have $\sum_{i=1}^{k-1} w(A_i) \leq (1 - \frac{1}{k}) \sum_{i=1}^{k} w(A_i) = 2(1 - \frac{1}{k})w(A)$

The $k-1$ cuts are identified as follows. Let B be the set of edges of T that connect across two of the sets $V_1, V_2, ..., V_k$. Consider the graph on vertex set

V and edge set B, and shrink each of the sets $V_1, V_2, ..., V_k$ to a single vertex. This shrunk graph must be connected (since T was connected). Throw edges away until a tree remains. Let $B' \subseteq B$ be the left over edges, $|B'| = k-1$. The edges of B' define the required $k-1$ cuts. Next, root this tree at V_k (recall that A_k was assumed to be the heaviest cut among the cuts A_i). This helps in defining a correspondence between the edges in B' and the sets $V_1, V_2, ..., V_{k-1}$: each edge corresponds to the set it comes out of in the rooted tree.

Suppose edge $(u, v) \in B'$corresponds to set V_i in this manner. The weight of a minimum u–v cut in G is $w'(u, v)$. since $A_i$is a u-v cut in G, $w(A_i) \geq w'(u, v)$.

Thus each cut among $A_1, A_2, ..., A_{k-1}$is at least as heavy as the cut defined in G by the corresponding edge of B'. This, together with the fact that C is the union of the lightest $k-1$ cuts defined by T, gives: $w(C) \leq \sum_{e \in B'} w'(e) \leq \sum_{i=1}^{k-1}$

$w(A_i) \leq (1 - \frac{1}{k}) \sum_{i=1}^{k} w(A_i) = 2(1 - \frac{1}{k})w(A)$